Web Application Firewall

Project Documentation

Steps 3 & 4: High-Level Design & Development Plan

Cybersecurity Project — Academic Year 2024/2025

April 2025

3. High-Level Design Document

3.0 Design Philosophy: Zero Trust & Security Principles

Before describing the architecture itself, it is important to establish the security philosophy that shapes every design decision in this WAF.

3.0.1 Zero Trust: Every Request Is Suspect Until Proven Safe

The WAF is built on a Zero Trust model. This means the system does not assume any request is safe simply because it comes from a known IP address, uses a valid certificate, or carries a session cookie. Every single request — regardless of origin — is treated as potentially malicious and must pass through the full inspection pipeline before being allowed through.

This is the opposite of a traditional perimeter model, where traffic inside the network is trusted by default. In our WAF:

- No request is forwarded to the origin server without passing every inspection stage
- Every decision (BLOCK or ALLOW) is logged, creating a complete audit trail
- Rules can be updated at runtime without restarting the WAF, so the system adapts **continuously**

3.0.2 Access Control: Who Can Access What

The WAF enforces two distinct layers of access control:

Traffic-Level Access Control — applied to every HTTP/HTTPS request passing through the WAF:

- Known malicious IPs are blocked immediately via the IP blocklist, before any other processing
- Rate limiting prevents any single source from overwhelming the protected server
- The Rule Engine enforces content-based access control: requests containing attack patterns are denied regardless of who sent them

Administrative Access Control — applied to the Admin Dashboard and REST API:

- All Admin API endpoints require a valid JWT (JSON Web Token) to access
- Admin passwords are stored as bcrypt hashes — never in plaintext

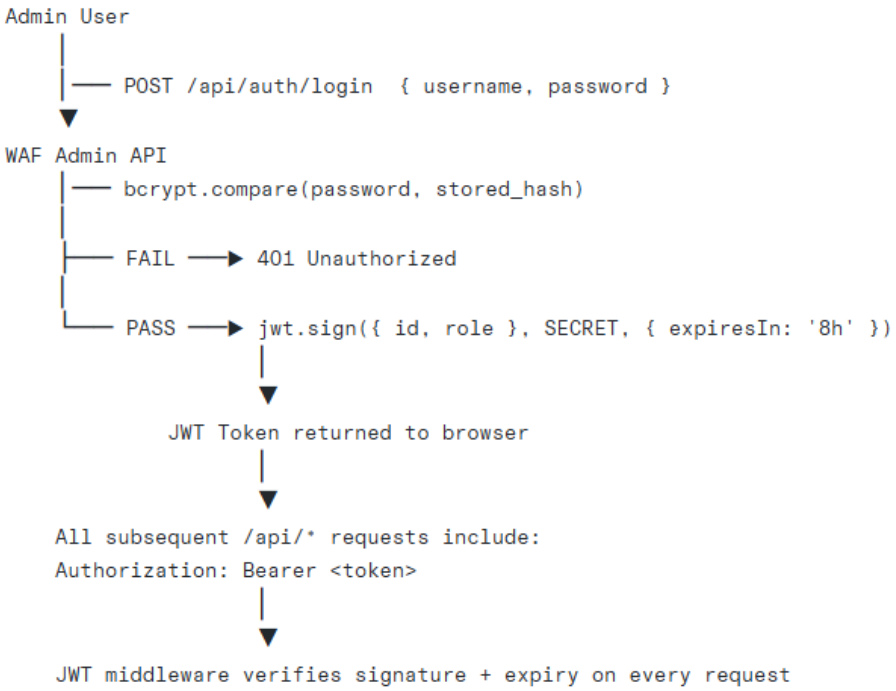
- The dashboard runs on a separate port from the WAF proxy, so it can be independently firewalled
- HTTPS is enforced on all admin traffic

3.0.3 Authentication Flow: How Identity Is Verified

For WAF traffic inspection, the WAF does not perform user authentication itself — that is the origin application's job. However, the WAF does verify the integrity of authentication-related data as it passes through:

- Cookie headers are parsed and inspected for injection attempts
- Missing CSRF tokens on state-changing requests (POST, PUT, DELETE) are flagged as a CSRF indicator
- Requests with forged or anomalous session tokens trigger protocol anomaly scoring

For Admin Dashboard access, the authentication flow works as follows:

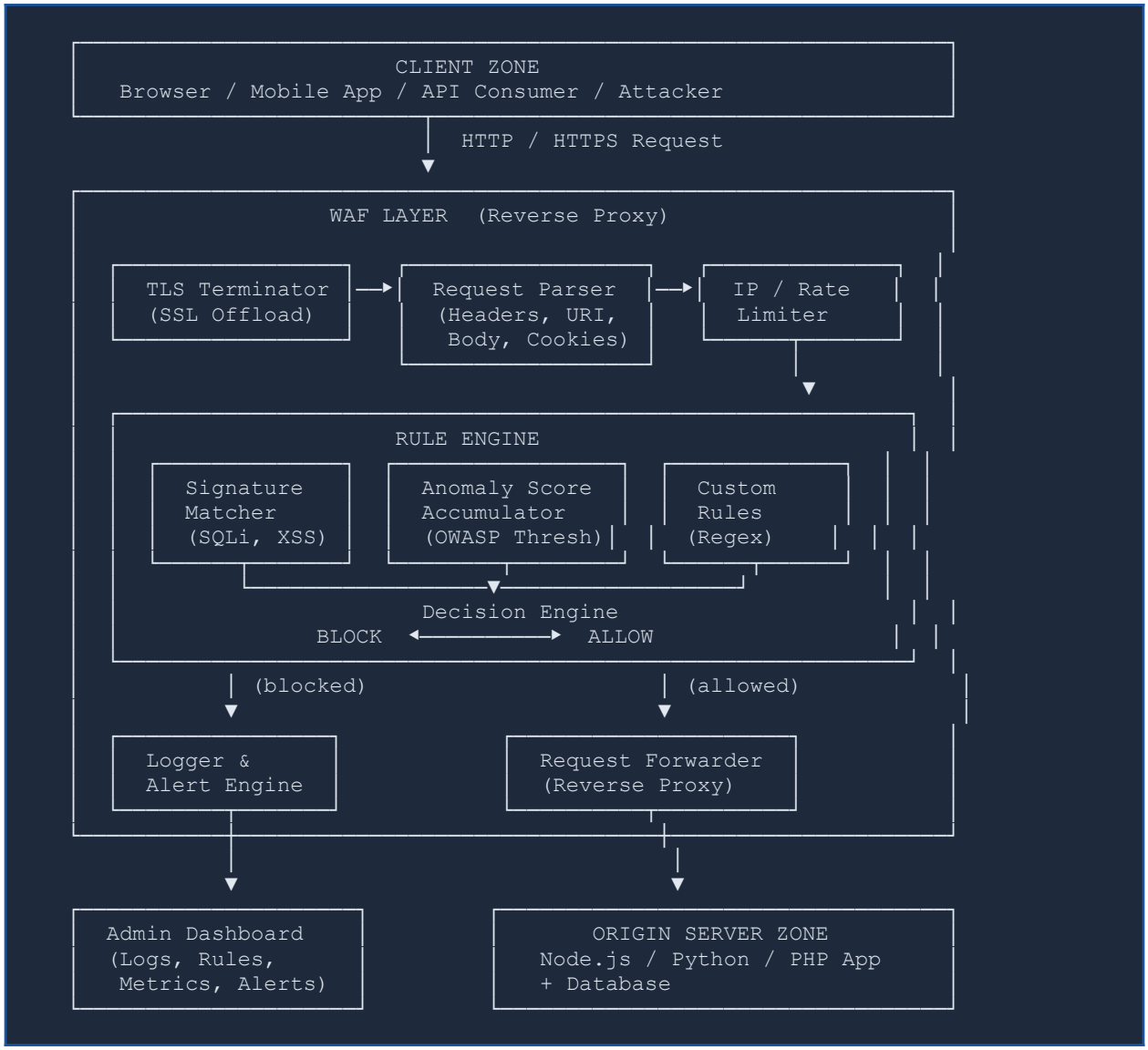


3. High-Level Design Document

3.1 System Architecture Overview

Our WAF is implemented as a standalone reverse proxy server that intercepts all HTTP/HTTPS traffic destined for the protected origin server. The architecture follows a pipeline pattern: each incoming request passes through a sequence of processing modules, and a final decision (BLOCK or ALLOW) is made before the request is forwarded or rejected.

The system is composed of five major zones: the Client Zone, the WAF Layer, the Origin Server Zone, the Persistence Layer (database), and the Admin Dashboard.



3.2 Component Description

3.2.1 TLS Terminator

Handles SSL/TLS offloading. All HTTPS connections from clients are decrypted at this layer, allowing the rest of the pipeline to operate on plaintext HTTP. This uses Node.js built-in TLS or the 'https' module. After inspection, traffic to the origin server can optionally be re-encrypted (TLS passthrough mode).

- Accepts HTTPS connections on port 443
- Decrypts using a self-signed or CA-issued certificate
- Passes plaintext IncomingMessage objects to the Request Parser

3.2.2 Request Parser

Decomposes each incoming HTTP request into its constituent parts for downstream analysis. The parser extracts and normalizes all request attributes before handing them to the Rule Engine.

Parsed Attribute	Description
HTTP Method	GET, POST, PUT, DELETE, PATCH — unusual methods are flagged
URI / Path	Full request path, decoded and normalized (URL-decode, Unicode normalize)
Query Parameters	Key-value pairs from the URL query string
Request Headers	All headers including Host, User-Agent, Referer, Content-Type
Cookies	Cookie header parsed into individual key-value pairs
Request Body	Raw body and parsed form data (application/x-www-form-urlencoded, JSON, multipart)

3.2.3 IP & Rate Limiter

This module performs two fast pre-checks before the more expensive rule evaluation:

- IP Blocklist Check: the source IP is checked against a configurable blocklist of known malicious IPs and CIDR ranges. Blocked IPs receive an immediate 403 response without entering the rule engine.
- Rate Limiting: a sliding-window counter tracks request frequency per IP. IPs exceeding the configured threshold (e.g., 100 requests per minute) are temporarily blocked and the event is logged.

3.2.4 Rule Engine

The Rule Engine is the core intelligence of the WAF. It evaluates all parsed request attributes against a set of security rules organized into categories. The engine uses an anomaly scoring

model: each rule match adds a score value, and the total is compared against a configurable threshold.

Rule Category	Examples of Patterns Detected	Score Weight	Scope
SQL Injection	UNION SELECT, OR 1=1, DROP TABLE, xp_cmdshell	5	URI, Body, Cookies
Cross-Site Scripting	<script>, onerror=, javascript:, eval(	5	URI, Body, Headers
Path Traversal	../, %2e%2e/, /etc/passwd, /proc/self	4	URI, Headers
Command Injection	; ls, && cat, | whoami, `id`	5	Body, URI
CSRF Indicators	Missing CSRF token on state-changing requests	3	Headers, Cookies
Protocol Anomalies	Malformed headers, invalid HTTP version, oversized fields	2	Headers
Suspicious User-Agent	Known scanner signatures: sqlmap, nikto, dirbuster	3	Headers

Decision logic: if total anomaly score ≥ threshold (default: 5), the request is BLOCKED and a 403 response is sent. If score < threshold, the request is forwarded to the origin server.

3.2.5 Request Forwarder

For allowed requests, this module acts as a standard HTTP reverse proxy. It forwards the original request (with headers preserved) to the configured origin server and relays the response back to the client. It optionally inspects responses for data leakage patterns (e.g., stack traces, database error messages).

3.2.6 Logger & Alert Engine

Every WAF decision — whether BLOCK or ALLOW — is asynchronously written to the persistence layer. For blocked events, the system can trigger real-time alerts via configurable channels.

- Structured JSON log entries persisted to SQLite (development) or PostgreSQL (production)
- Log fields: timestamp, source IP, method, URI, matched rule(s), anomaly score, action taken
- Alert channels: console output, log file, and a WebSocket push to the live dashboard

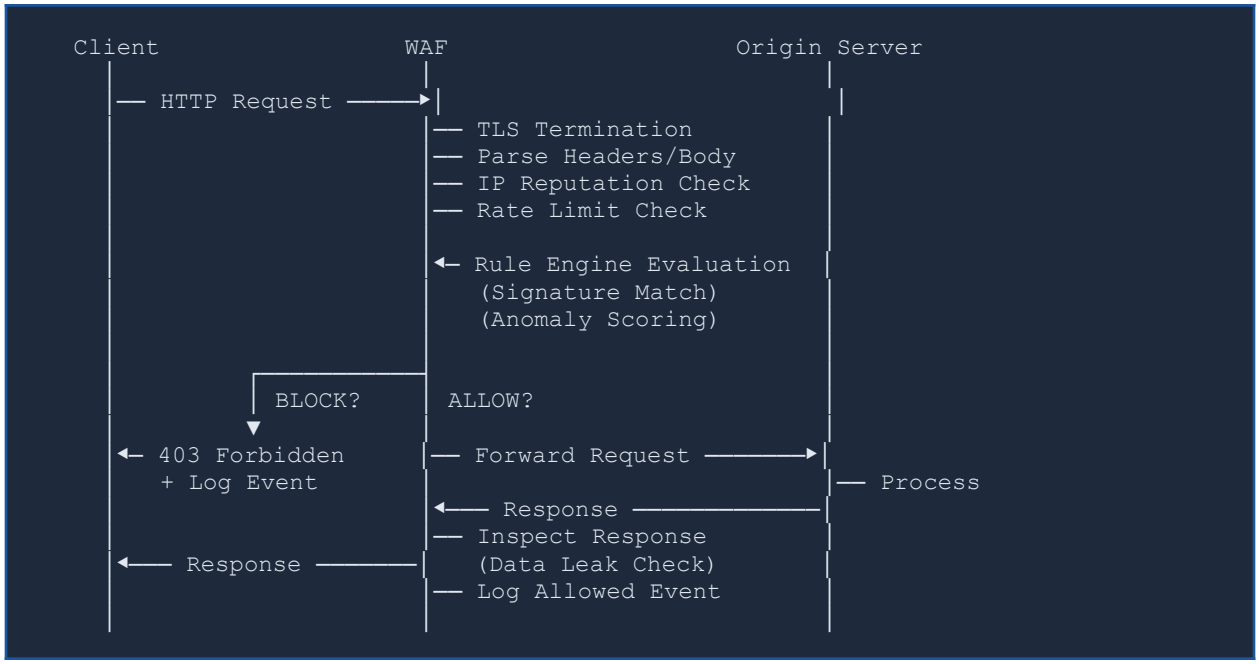
3.2.7 Admin Dashboard

A web-based single-page application (React) that provides real-time visibility into WAF operations and allows administrators to manage rules and configuration.

- Live feed of blocked and allowed requests with filtering by IP, rule, and time range
- Aggregated metrics: total requests, block rate, top attacked endpoints, top source IPs
- Rule management: create, enable/disable, edit, and delete rules without restarting the WAF
- IP blocklist management: add/remove individual IPs or CIDR ranges
- Attack category breakdown charts (SQLi vs XSS vs Path Traversal, etc.)

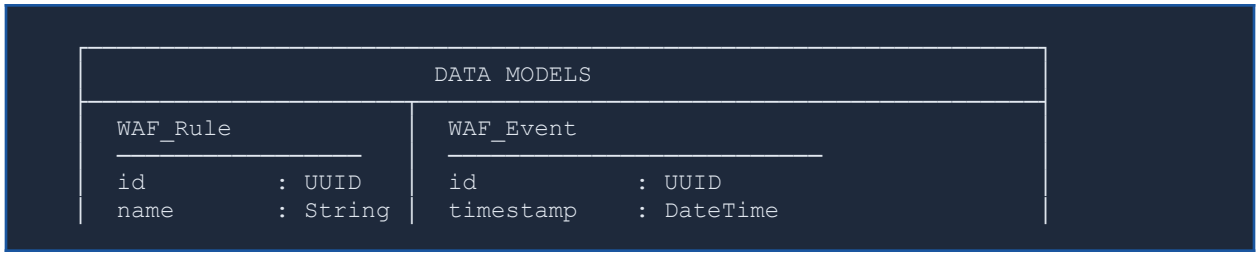
3.3 Request Processing Flow

The following sequence diagram illustrates the complete lifecycle of an HTTP request through the WAF, from client initiation to final response:



3.4 Data Models

The WAF persists two primary entities: WAF_Rule (the security ruleset) and WAF_Event (the audit log of all decisions made).



```
┌──────────────────────┐  ┌──────────────────────────────────────────┐
│  description: String │  │  src_ip       : String                   │
│  pattern     : Regex │  │  method       : String                   │
│  category    : Enum  │  │  uri          : String                   │
│    SQLi / XSS /      │  │  matched_rule : FK → WAF_Rule            │
│    PathTraversal /   │  │  anomaly_score: Integer                  │
│    CmdInject / CSRF  │  │  action       : Enum (BLOCK/ALLOW/LOG)   │
│  score       : Int   │  │  payload_snip : String (first 512 chars) │
│  enabled     : Bool  │  │  response_code: Integer                  │
│  created_at  : Date  │  │  geo_country  : String                   │
└──────────────────────┘  └──────────────────────────────────────────┘
```

3.5 Key Design Decisions

Design Decision	Choice Made	Rationale
Architecture pattern	Reverse Proxy (not inline appliance)	Works without modifying origin app; easy to deploy in front of any server
Detection model	Anomaly scoring (not binary block-on-match)	Reduces false positives; allows tuning of sensitivity via threshold
Rule storage	Database (not flat config file)	Enables runtime rule updates via dashboard without WAF restart
Logging	Asynchronous, non-blocking	Logging must not add latency to the request processing pipeline
Dashboard	Separate React SPA, not embedded	Clean separation of concerns; dashboard can be secured independently
Language	Node.js (backend) + React (frontend)	Event-driven I/O model ideal for a high-concurrency proxy; same JS ecosystem

3.6 Interfaces & Exchanged Data

3.6.1 Client ↔ WAF

Standard HTTP/1.1 or HTTP/2 protocol. The client is unaware of the WAF's presence — it communicates exactly as it would with the origin server. Blocked requests receive a structured JSON error response:

```
// BLOCKED response body (JSON)
{
  "error": "Forbidden",
  "code": 403,
  "message": "Request blocked by WAF security policy.",
```

```
  "ref":  "EVT-20250418-0047"
}
```

3.6.2 WAF ↔ Origin Server

The WAF forwards the original request with all original headers preserved, plus two additional headers injected for tracing:

```
X-WAF-Request-ID: <uuid>          // Unique ID for this request
X-WAF-Anomaly-Score: <integer>    // Score computed by rule engine
X-Forwarded-For: <client-ip>      // Original client IP for origin logs
```

3.6.3 WAF ↔ Admin Dashboard (REST API)

Method	Endpoint	Auth	Description
GET	/api/events	JWT	Paginated list of WAF events with filters
GET	/api/events/:id	JWT	Full detail of a single event
GET	/api/rules	JWT	List all WAF rules
POST	/api/rules	JWT	Create a new rule
PUT	/api/rules/:id	JWT	Update or enable/disable a rule
DELETE	/api/rules/:id	JWT	Delete a rule
GET	/api/stats	JWT	Aggregated metrics (counts, block rate, top IPs)
GET	/api/blocklist	JWT	List of blocked IPs/CIDRs
POST	/api/blocklist	JWT	Add IP or CIDR to blocklist
DELETE	/api/blocklist/:ip	JWT	Remove from blocklist
WS	/ws/events	JWT	Live event stream pushed to dashboard

4. Tools, Technologies & Development Phases

4.0 Why These Tools Were Chosen

This section explains not just what tools are used, but why each one was selected for a security-focused project.

Node.js was chosen as the primary runtime because its event-driven, non-blocking I/O model is ideal for a reverse proxy that must handle many concurrent connections without blocking the processing pipeline for any single request. In a WAF, adding latency is unacceptable — Node.js allows logging, rule evaluation, and request forwarding to happen asynchronously and efficiently.

Python is referenced in the testing layer (via tools like OWASP ZAP, which is Java-based but integrates with Python scripting) because of its strength in security tooling and scripting attack scenarios.

SQLite is used in development because it requires zero configuration and is embedded directly in the process — ideal for a course project and local testing. The design explicitly supports migration to PostgreSQL for production use, which handles high concurrency better.

OWASP ZAP was selected as the primary attack simulation tool because it is the industry-standard open-source web vulnerability scanner, maintained by the OWASP Foundation. It can generate realistic SQLi, XSS, and path traversal payloads automatically, making it ideal for testing whether the WAF actually blocks real attack patterns.

4.1 Technology Stack

4.1.1 Backend — WAF Engine

Technology	Version	Role in the Project
Node.js	20 LTS	Runtime for the WAF reverse proxy and REST API server
Express.js	4.x	HTTP framework for the Admin REST API
http-proxy	1.x	Core reverse proxy forwarding for allowed requests
ws (WebSocket)	8.x	Real-time event push from WAF engine to Admin Dashboard
jsonwebtoken	9.x	JWT generation and verification for Admin API authentication
bcrypt	5.x	Admin password hashing for secure credential storage
better-sqlite3	9.x	Embedded SQLite database for rules and event logs
helmet	7.x	Sets secure HTTP headers on all API responses

rate-limiter-flexible	5.x	Token-bucket rate limiting per source IP
dotenv	16.x	Environment variable management (.env files)

4.1.2 Frontend — Admin Dashboard

Technology	Version	Role in the Project
React	18.x	Component-based SPA framework for the Admin Dashboard
Vite	5.x	Development server and build tool (fast HMR)
Tailwind CSS	3.x	Utility-first CSS framework for dashboard styling
Recharts	2.x	Charts for request volume, attack categories, block rate
React Query	5.x	Server state management and API polling for live data
React Router	6.x	Client-side routing between dashboard pages
Axios	1.x	HTTP client for Admin REST API calls

4.1.3 Testing & Attack Simulation Tools

Tool	Purpose
OWASP ZAP	Automated vulnerability scanner used to generate realistic attack traffic for WAF testing
sqlmap	SQL injection payload generator — used to validate that the WAF correctly blocks SQLi attempts
curl / HTTPie	Manual HTTP request crafting for unit-level rule testing
Jest	Unit testing framework for rule engine logic (Node.js)
Supertest	Integration testing of the Admin REST API endpoints
Artillery	Load testing tool to measure WAF throughput and latency under high concurrency
Wireshark	Network-level traffic capture to verify TLS termination and header injection

4.1.4 DevOps & Development Infrastructure

Tool	Purpose

Docker + Docker Compose	Containerize WAF, Dashboard, and a vulnerable target app (DVWA) for demo environment
Git + GitHub	Source control, branching strategy, and pull-request-based code review
GitHub Actions	CI pipeline: linting, unit tests, and Docker image build on every push
ESLint + Prettier	Code style enforcement across the JavaScript codebase
Postman	API documentation and manual testing of Admin REST endpoints
DVWA	Damn Vulnerable Web App — used as the protected origin server to demonstrate WAF effectiveness

4.2 Development Phases

The project is organized into five sequential phases over an estimated 8-week timeline. Each phase has defined deliverables and exit criteria before the next phase begins.

#	Phase	Duration	Deliverables & Exit Criteria
1	Foundation & Core Proxy	Week 1–2	Working HTTP/HTTPS reverse proxy that can forward requests to an origin server. Project scaffold, Git repo, Docker Compose with DVWA target, environment configuration.
2	Rule Engine & Detection	Week 3–4	Rule engine with SQLi, XSS, Path Traversal, Command Injection, and Protocol Anomaly detection. Anomaly scoring model. Unit tests achieving >80% coverage on rule logic.
3	Logging, API & Persistence	Week 5	SQLite schema created. All WAF events persisted asynchronously. Admin REST API (rules + events + stats + blocklist) with JWT authentication. Postman collection documented.
4	Admin Dashboard	Week 6	React dashboard with live event feed (WebSocket), attack charts, rule CRUD interface, and IP blocklist management. Fully functional against the running WAF backend.
5	Testing & Demo Hardening	Week 7–8	Full attack simulation using OWASP ZAP and sqlmap against DVWA through the WAF. Performance test with Artillery. Bug fixes, final documentation, and demo preparation.

4.3 Detailed Phase Breakdown

Phase 1 — Foundation & Core Proxy (Weeks 1–2)

1. Initialize Node.js project with Express and configure ESLint + Prettier

2. Set up Docker Compose with three services: waf, dashboard, and dvwa (target)
3. Implement basic HTTP reverse proxy using http-proxy-middleware
4. Add HTTPS support with a self-signed certificate using Node's tls module
5. Inject X-Forwarded-For and X-WAF-Request-ID headers on forwarded requests
6. Write integration tests verifying that clean requests pass through correctly

Phase 2 — Rule Engine & Detection (Weeks 3–4)

7. Design the Rule schema (id, name, category, pattern, score, enabled)
8. Implement the Request Parser module extracting all HTTP attributes
9. Build the Signature Matcher: iterate rules, apply regex patterns to each attribute
10. Implement the Anomaly Score Accumulator with configurable threshold
11. Implement the Decision Engine: BLOCK (return 403) or ALLOW (forward)
12. Add IP blocklist check and sliding-window rate limiter
13. Seed the rule database with patterns for SQLi, XSS, Path Traversal, CMD Injection
14. Write unit tests for each rule category using crafted malicious payloads

Phase 3 — Logging, API & Persistence (Week 5)

15. Create SQLite schema: waf_rules and waf_events tables
16. Implement async Logger module writing events without blocking the request pipeline
17. Build the Admin REST API with Express: all /api/* endpoints
18. Implement JWT authentication middleware for all Admin API routes
19. Add WebSocket server broadcasting new events to connected dashboard clients
20. Test all API endpoints with Supertest and document with Postman

Phase 4 — Admin Dashboard (Week 6)

21. Scaffold React + Vite + Tailwind CSS project
22. Implement login page with JWT-based authentication flow
23. Build the Live Events page with WebSocket connection and auto-updating table
24. Build the Statistics page with Recharts: block rate trend, attack category pie, top IPs
25. Build the Rule Manager page: list, create, edit, enable/disable, and delete rules
26. Build the IP Blocklist page: add/remove IPs and CIDR ranges
27. End-to-end test of the full stack: attack → WAF block → dashboard update

Phase 5 — Testing & Demo Hardening (Weeks 7–8)

28. Run OWASP ZAP full scan against DVWA through the WAF and analyze blocked vs missed attacks
29. Run sqlmap against a login form behind the WAF and verify 100% SQLi block rate
30. Execute Artillery load test (500 concurrent users, 2 minutes) and measure p99 latency
31. Fix any false positives or false negatives discovered during testing
32. Write final project report and prepare live demonstration environment
33. Record a short video demo showing: attack attempt → WAF block → dashboard visualization

4.4 Project Repository Structure

```
waf-project/
├── waf/                          # WAF Engine (Node.js)
│   ├── src/
│   │   ├── proxy.js              # Core reverse proxy + TLS termination
│   │   ├── parser.js             # HTTP request parser
│   │   ├── ruleEngine.js         # Signature matcher + anomaly scoring
│   │   ├── ipFilter.js           # IP blocklist + rate limiter
│   │   ├── logger.js             # Async event logger
│   │   ├── db.js                 # SQLite connection + schema
│   │   └── api/                  # Admin REST API (Express routes)
│   │       ├── events.js
│   │       ├── rules.js
│   │       ├── stats.js
│   │       └── blocklist.js
│   ├── tests/                    # Jest unit + Supertest integration tests
│   ├── seeds/                    # Initial rule set (SQLi, XSS, Path
Traversal…)
│   ├── certs/                    # Self-signed TLS certificate
│   └── package.json
│
├── dashboard/                    # Admin Dashboard (React + Vite)
│   ├── src/
│   │   ├── pages/                # Events, Stats, Rules, Blocklist pages
│   │   ├── components/           # Shared UI components
│   │   ├── hooks/                # Custom hooks (useWebSocket, useApi)
│   │   └── App.jsx
│   └── package.json
│
├── docker-compose.yml            # WAF + Dashboard + DVWA orchestration
├── .github/workflows/ci.yml      # GitHub Actions CI pipeline
└── README.md                     # Setup + demo instructions
```

4.5 Risk Assessment & Mitigations

Risk	Likelihood	Mitigation Strategy
High false-positive rate blocking legitimate traffic	High	Tunable anomaly threshold; Detection-Only mode for initial deployment; extensive testing with clean traffic

Rule engine performance bottleneck at high load	Medium	Async processing; compiled regex objects cached at startup; Artillery load testing in Phase 5
Evasion via URL encoding or double encoding	Medium	URL normalization and multi-pass decoding in the Request Parser before pattern matching
SQLite not sufficient for high concurrency	Low	WAL mode enabled; easy migration path to PostgreSQL via Knex.js query builder
Admin dashboard exposed without proper auth	Low	JWT + bcrypt; dashboard served on separate port; HTTPS enforced